SMARTPHONE : OUKITEL

APPLICATION ANDROID nomée : Python For Beginners

Python Master Class (by VDA Team)

Part 3

Continuation Lines :

When a single logical line of code is too long, it can be split into multiple physical lines using parentheses, brackets, or braces to group expressions without needing a backslash :

```
total = (item_one + item_two + item_three
+
        Item_four + item_five)
```

This approach is often preferred for its readability.

Variables and Naming Conventions :

Python variables are dynamically typed, meaning you don't need to declare their type before assigning a value. However, choosing meaningful names and following naming conventions is crucial for maintaining clean and understandable code.

Naming Rules and Conventions :

Variable names must start with a letter (a-z, A-Z) or an underscore (_).

Subsequent characters can include letters, digits (0-9), and underscores.

Variable names are case-sensitive :

`Variable`, `variable`, and `VARIABLE` are distinct identifiers.

Avoid using Python reserved keywords like `if`, `else`, `while`, etc., as variable names.

Following the PEP8 naming conventions is recommended :

Snake_case for variable names and functions (e.g., `student_name`, `calculate_total`).

UpperCamelCase for class names (e.g., `MyClass`, `EmployeeData`).

Constants should be written in UPPERCASE_WITH_UNDERSCORES (e.g., `MAX_LIMIT`, `PI_VALUE`).

Examples of Variable Naming :

```
# Good examples

Student_name = "Alice"

total_score = 95

MAX_LIMIT = 100


# Bad examples

X = 5 # Unclear what 'x' represents

T = 30 # Hard to understand at first glance
```

Data Types and Type Conversion :

Python supports several built-in data types that are used to represent different kinds of data. Understanding these types and how to convert between them is essential for mastering Python syntax.

Basic Data Types :

The most commonly used data types in Python include :

Integers : Whole numbers without a decimal point, e.g., `10`, `-42`.

Floats : Numbers with a decimal point, e.g., `3.14`, `-0.001`.

Strings : A sequence of characters, enclosed in single or double quotes, e.g., `"Hello, World !"`, `'Python'`.

Booleans : Represents `True` or `False` values.

Lists : Ordered collections of items, e.g., `[1, 2, 3, "apple"]`.

Tuples : Immutable ordered collections, e.g., `(1, 2, 3)`.

Sets : Unordered collections of unique items, e.g., `{1, 2, 3}`.

Dictionaries : Collections of key-value pairs, e.g., `{"name" : "Alice", "age" : 25}`.

Type Conversion :

Python provides several built-in functions to convert between types :

```
# Converting a string to an integer

Num_str = "100"

Num_int = int(num_str)
```

```
# Converting an integer to a float

Num_float = float(num_int)


# Converting a list to a tuple

Num_list = [1, 2, 3]

Num_tuple = tuple(num_list)


print(num_int)   # Output: 100

print(num_float)   # Output: 100.0

print(num_tuple)   # Output: (1, 2, 3)
```